
O&M Cane Training

Technical overview — how the app is built, what each part does, and how data and audio flow through it. Written for the team: developer, reviewer, and anyone picking the project up later.

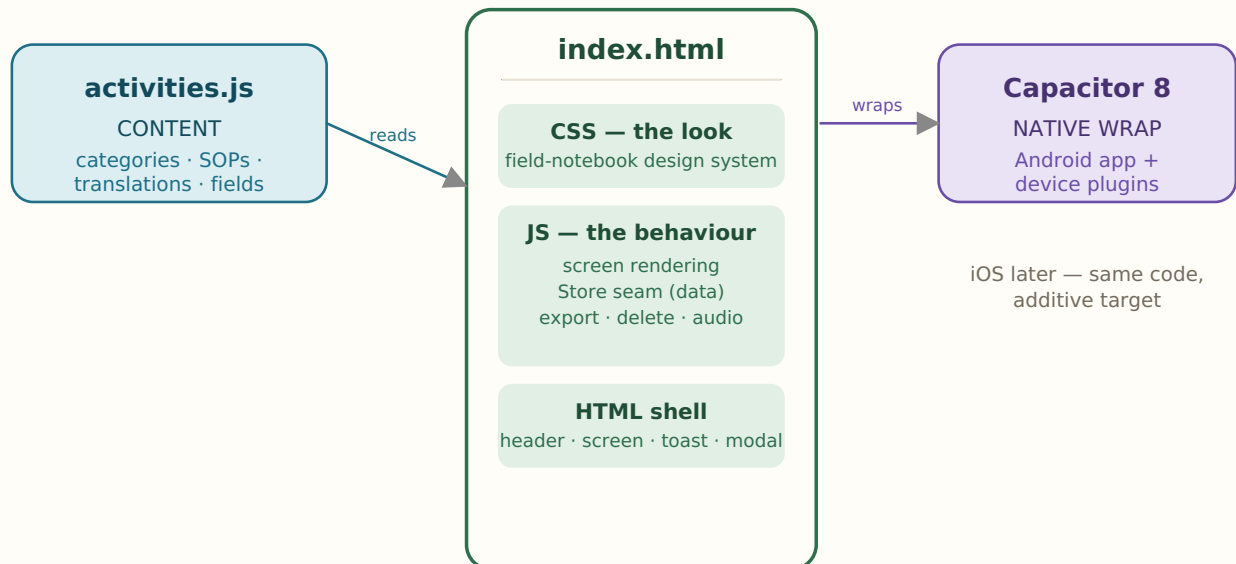
v0 · plain HTML/CSS/JS · Capacitor 8 (Android) · no bundler · offline-first

1. The big picture — one file, three concerns
2. The screen flow — how a teacher moves through the app
3. The Store seam — where all data lives, and why it's clever
4. The data model — children, records, and how they link
5. Saving a result — the write-verify path end to end
6. Deletion — three tiers of "remove"
7. CSV export — the data escape hatch
8. Multi-language SOP audio — the TTS pipeline
9. Who edits what — the content boundary
10. Function map — quick reference

01 ARCHITECTURE

The big picture — one file, three concerns

The whole app is essentially `index.html` — one file that holds the look (CSS), the behaviour (JavaScript), and the screens. It reads its content from `activities.js`, and on a phone it's wrapped by Capacitor into a native Android app. There is deliberately **no build step**: editing content means editing a text file and refreshing.



The separation that makes the project maintainable: content people touch the blue box; engineers touch the green box.

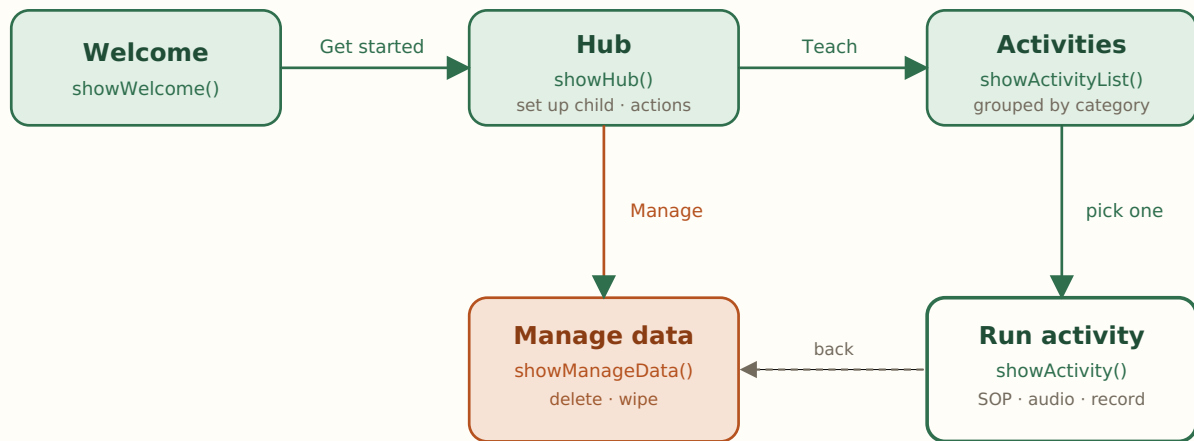
THE LOAD-BEARING DECISION

No bundler. A build step (like Vite) would break the workflow where a non-coder edits `activities.js` and just refreshes to see the change. Everything downstream — the Store seam, the derived audio paths — is shaped to preserve that.

02 NAVIGATION

The screen flow — how a teacher moves through the app

There is one container (`#screen`) and JavaScript repaints it for each screen. `paint()` swaps the HTML and manages the entrance animation and focus. Five screens:



On every screen the header shows:

- a back button (hidden on Welcome/Hub) · a home icon · the screen's breadcrumb
- The category colour theme (*themeFor()*) switches when you enter an activity, tinting the run screen.

Welcome → Hub is the spine. Activities and Run hang off "Teach"; Manage-data is deliberately off to the side so destructive controls aren't on the main path.

03 STORAGE

The Store seam — where all data lives, and why it's clever

Every piece of saved data — children, results, settings — goes through one object called **Store**. Nothing else in the app talks to storage directly. That single choke point is why swapping the storage backend (browser → native phone storage) was a ~4-line change, and it's where the project's most careful engineering lives.

The problem it solves

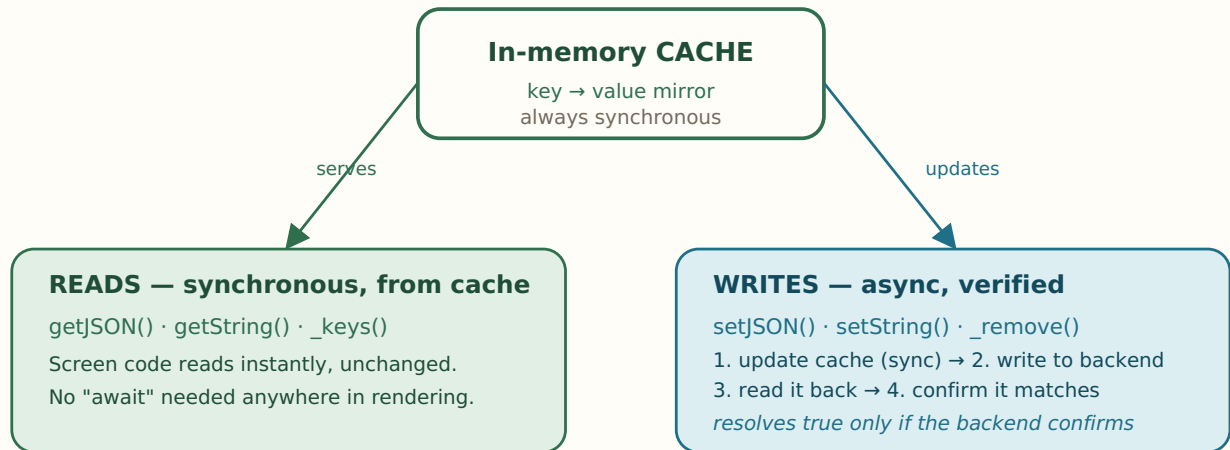
On a phone, Capacitor's native storage is **asynchronous** — every read/write returns a promise you have to wait for. But the screen-drawing code reads data **synchronously** while building HTML. Rewriting every screen to "await" its reads would be huge and fragile. So the Store keeps an in-memory **cache** that mirrors the backend:

BOOT — once, at startup (Store.init)

Detect backend: native phone? → Capacitor Preferences. Plain web? → localStorage.

Load the ENTIRE backend into the in-memory cache. The first screen waits for this to finish.

Until the cache is filled, reads would be empty — so boot blocks on it.



Reads are instant (cache). Writes go to the backend AND read back to verify — because in a phone WebView a write can fail silently, which is the worst outcome for children's data.

WHY READ-BACK VERIFICATION MATTERS

Inside a native WebView, a storage write can fail silently (quota, OS error). For a children's-data tool, vanishing data is the cardinal sin. So every write is read back and compared; if it doesn't match, the save returns `false` and the teacher sees "Could not save — export your data and try again" instead of losing the record quietly.

To change storage backends (e.g. add cloud sync later), you only edit the four `_backend*` functions and the detection in `init()`. The cache, the verify logic, and everything that calls Store stay exactly as they are.

04 DATA

The data model — children, records, and how they link

Two kinds of saved data, stored under different keys, joined by a child's id.

CHILD PROFILE

key: profiles (array of these)

id — unique, e.g. "c1abc..."

name

dob — age is DERIVED, never stored

height · weight

dominantHand

photo — optional, low-res

capturedOn

profileId → id

SESSION RECORD

key: rec_<activityId> (array)

profileId — links to a child

student — name snapshot

when — timestamp

values — what was recorded:

count / result / checkbox / notes
(fields defined per activity
in activities.js)

Each result points back to a child. Delete a child → all their records are swept too.

Profiles are stored once under the `profiles` key. Each activity's results live under their own `rec_<activityId>` key, linked back by `profileId`.

STORAGE KEY

HOLDS

`profiles`

Array of all child profiles

`activeProfile`

Id of the currently-selected child

`rec_<activityId>`

Array of session records for one activity

`audioLang`

Chosen SOP audio language (hi / ta / bn)

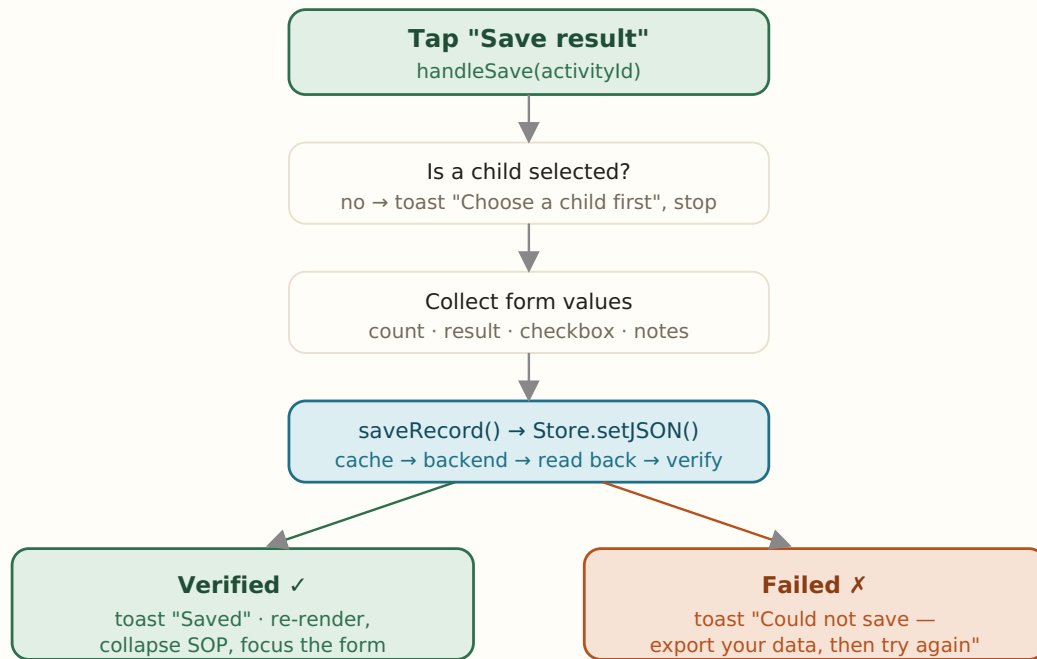
`welcomeSeen` · `studentName`

Legacy flags, kept for compatibility

05 THE WRITE PATH

Saving a result — the write-verify path end to end

When a teacher taps "Save result," here's exactly what happens:



The fork at the bottom is the whole point: a save only reports success once the backend has confirmed it. A silent failure becomes a visible warning.

06 DELETION

Deletion — three tiers of "remove"

Because this is children's disability data, deletion is a real feature, not a debug path. Three levels, each confirmed through an in-app modal (never the browser's `confirm()`, which is unreliable inside a `WebView`), each nudging toward a CSV backup first.

TIER	FUNCTION	WHAT IT REMOVES	WHERE
One result	<code>deleteRecord()</code>	A single session record	Run screen
One child	<code>deleteProfile()</code>	The child <i>and</i> every record linked to them	Manage data
Everything	<code>clearAllData()</code>	All children + all records (keeps the welcome flag)	Manage data — danger zone, two confirms

DESIGN-AUDIT FIX APPLIED

All three used the browser's `window.confirm()`, which can be suppressed inside an Android `WebView` — an irreversible delete could fire without its guard. They now use an in-app `confirmModal()`: focus-trapped, Escape/backdrop cancels, focus opens on "Cancel," styled in the app's own colours. More reliable and more accessible than the native dialog.

07 EXPORT

CSV export — the data escape hatch

Until cloud sync exists, CSV export is how data leaves the device so it can't be lost.

`exportCSV()` gathers every record across all activities, joins each one to its child's demographics, and builds a single spreadsheet.

- `gatherAllRecords()` walks every `rec_` key and attaches the matching child's age (derived), DOB, height, weight, hand.
- `buildCSV()` writes proper CSV: quotes/escapes special characters, and prepends a UTF-8 BOM so Excel reads Indian-language names correctly.
- On a phone it writes the file to app-private storage and opens the Android **share sheet** (save to Files/Drive/email). On the web it's a normal download.

DESIGN-AUDIT FIX APPLIED

The native export wrote to `Directory.Cache`, which the OS can evict before the share sheet finishes on low-storage devices. Changed to `Directory.Data` — app-private, not evictable, no permission needed, and keeps children's data out of shared storage.

08 AUDIO

Multi-language SOP audio — the TTS pipeline

An activity's instructions (the SOP) can be narrated in Hindi, Tamil, or Bengali. This is a two-stage pipeline: audio is **generated ahead of time** on a computer (never in the app, never needing the internet in the classroom), then **played offline** in the app via a language switcher.

THE KEY INSIGHT THAT SHAPED THIS

Text-to-speech does not translate — it only speaks the text you give it. To get Hindi audio you need Hindi *text* first. So the content team provides translated SOP text (a native speaker checks medical-adjacent instructions); the generator only ever speaks that provided text — never an unreviewed machine translation.

Stage 1 — generation (on the Mac, run by the developer)



API key read from a local .env file — never in code, never shipped in the app, never committed to GitHub.

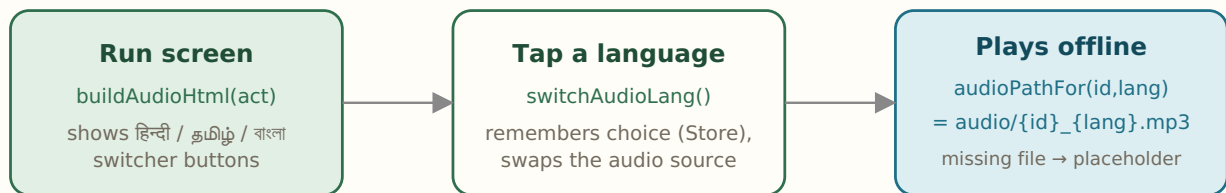
Files named by a fixed convention: audio/{activityId}_{lang}.mp3

Run once per activity; re-runs skip files that already exist.

The generator reads the team's translated text and produces one MP3 per language.

The file name is a contract with the app.

Stage 2 — playback (in the app, used by the teacher)



The app computes the file name from the activity id + language — so no filenames are stored in activities.js.

The switcher reuses the same segmented-control component as the result picker. A language with no generated file degrades gracefully to the placeholder.

09 BOUNDARIES

Who edits what — the content boundary

The project's central discipline: content people and engineers touch different files and never block each other.

ROLE	TOUCHES	DOES
Content team	<code>activities.js</code> only	Edits activity text, SOPs, and adds <code>sopTranslations</code> . Plain text between quotes; no code.
Developer	<code>index.html</code> , <code>generate-audio.js</code>	Behaviour, design, audio generation, the build & release loop.
Legal / compliance	— (process, not code)	Owens consent. The app records a consent flag; it doesn't decide policy.
O&M lead	— (review)	Signs off the scoring rubric and a few UX decisions.

TO ADD AUDIO TO ANOTHER ACTIVITY

The content team adds a `sopTranslations` block to that activity in `activities.js` (one line per SOP step, per language, in the same order). The developer then runs `generate-audio.js --only <id>` and the build loop. No `index.html` change is ever needed — the switcher appears automatically for any activity that has translations.

10 REFERENCE

Function map — quick reference

AREA	FUNCTIONS
Boot & storage	<code>Store.init</code> · <code>getJSON</code> · <code>getString</code> · <code>setJSON</code> · <code>setString</code> · <code>_remove</code>
Screens	<code>showWelcome</code> · <code>showHub</code> · <code>showActivityList</code> · <code>showActivity</code> · <code>showManageData</code> · <code>paint</code>
Profiles	<code>loadProfiles</code> · <code>upsertProfile</code> · <code>profileById</code> · <code>getActiveProfile</code> · <code>selectChild</code> · <code>handleProfileSave</code>
Records	<code>loadRecords</code> · <code>saveRecord</code> · <code>handleSave</code> · <code>renderRecord</code>
Deletion	<code>deleteRecord</code> · <code>deleteProfile</code> · <code>clearAllData</code> · <code>confirmModal</code> · <code>confirmDeleteChild</code> · <code>confirmClearAll</code> · <code>confirmDeleteRecord</code>
Export	<code>exportCSV</code> · <code>gatherAllRecords</code> · <code>buildCSV</code> · <code>csvCell</code>
Audio	<code>buildAudioHtml</code> · <code>switchAudioLang</code> · <code>audioPathFor</code> · <code>getAudioLang</code> · <code>audioMissing</code>
Theme & icons	<code>themeFor</code> · <code>catColor</code> · <code>catIcon</code>
Helpers	<code>ageFromDOB</code> · <code>downscaleImage</code> · <code>esc</code> · <code>toast</code>

This document describes the app as of the multi-language audio milestone. The storage seam, three-page flow, deletion tiers, and CSV export are settled; the audio pipeline is proven for one activity and awaits the content team's real translations for the rest.